eBPF'26 · 개별연구 9주차 논문 리뷰

Kunai

Toward a Verifier-Safe Layered DSL for Nested Encapsulation Packet Filtering on eBPF

Takeru Hayasaka et al.

26.10.29 박수현

목차

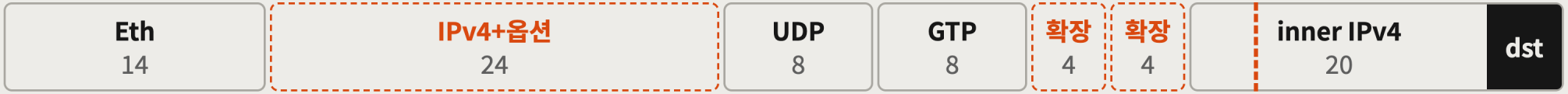
1	배경 및 문제 정의	가변 헤더 · 중첩 캡슐화 · 기존 연구
2	제안 기법	Kunai 구조 · verifier-safe 코드 생성
3	평가	명령어 수 · 처리율
4	추가 실험	veth · netkit 패킷 헤더 비교

1. 배경 및 문제 정의: 패킷 헤더 구조

기본 구조 — dst = 66



가변 길이 · 가변 개수 헤더 — dst = 78



중첩 캡슐화 — 같은 이름 헤더 IPv4 × 2



① 가변 길이·가변 개수 헤더 등장

컴파일 타임 오프셋 계산 불가

② 중첩 캡슐화 등장

같은 이름 헤더(outer·inner IPv4) 구분 불가

1. 배경 및 문제 정의: 기존 연구 pcap-filter



① cBPF 루프 부재

가변 개수 구조(GTP 확장 헤더 체인 · TCP 옵션) 반복 처리 불가

② 언어 문법 부재

중첩 구조의 같은 이름 헤더(outer · inner IPv4) 구분 불가

1. 배경 및 문제 정의: 기존 연구 P4-to-eBPF



① 가변 길이 미지원

컴파일러에서 런타임 가변 길이 접근 미지원

② 가변 개수 제한

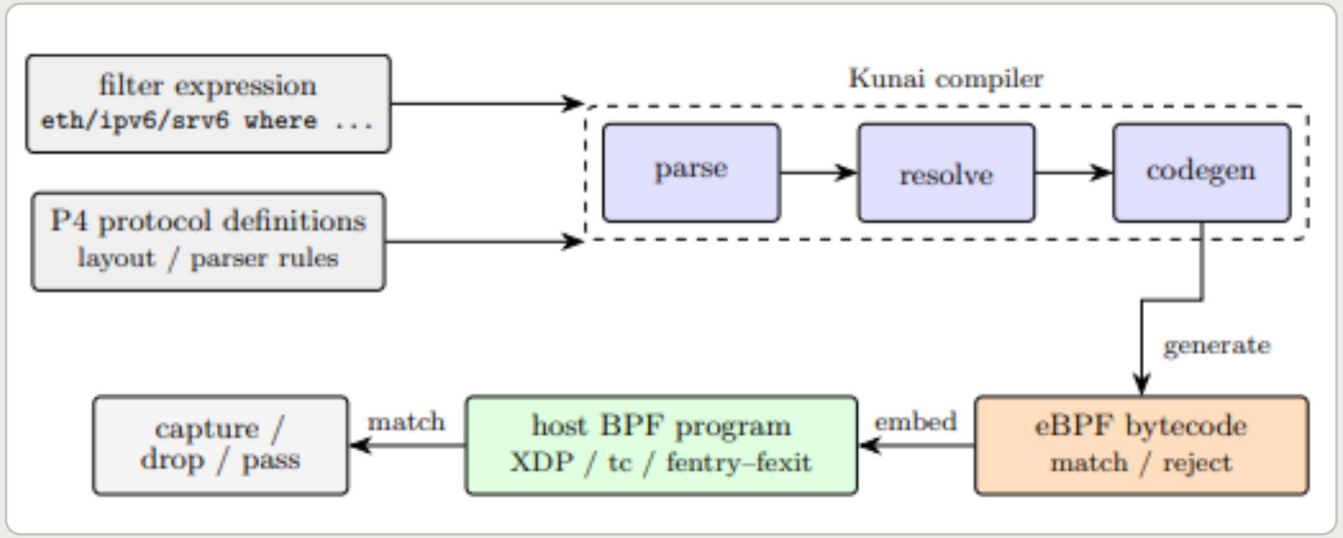
bpf_loop 같은 런타임 반복 기능 미지원

고정 구조만 처리 가능, 대신 패킷 수정 가능

필터(논문 주제)보다 방화벽 · 포워딩 엔진에 적합

2. 제안 기법: Kunai 개요

Kunai 구조 (Fig. 1)



입력

① 필터식 · 사용자 입력

② P4 프로토콜 subset (헤더 · 파서만)

컴파일러

verifier-safe eBPF 바이트코드 생성

실행

호스트 BPF 프로그램에 삽입

XDP · tc · fentry/fexit → 캡처 · 드롭 · 통과

2. 제안 기법: 해결 과제

3가지 문제를 verifier-safe 바이트코드로 인식하는 방법

해결해야 할 문제

① 가변 길이 헤더

헤더 시작 오프셋을 컴파일 타임에 모름

② 가변 개수 헤더

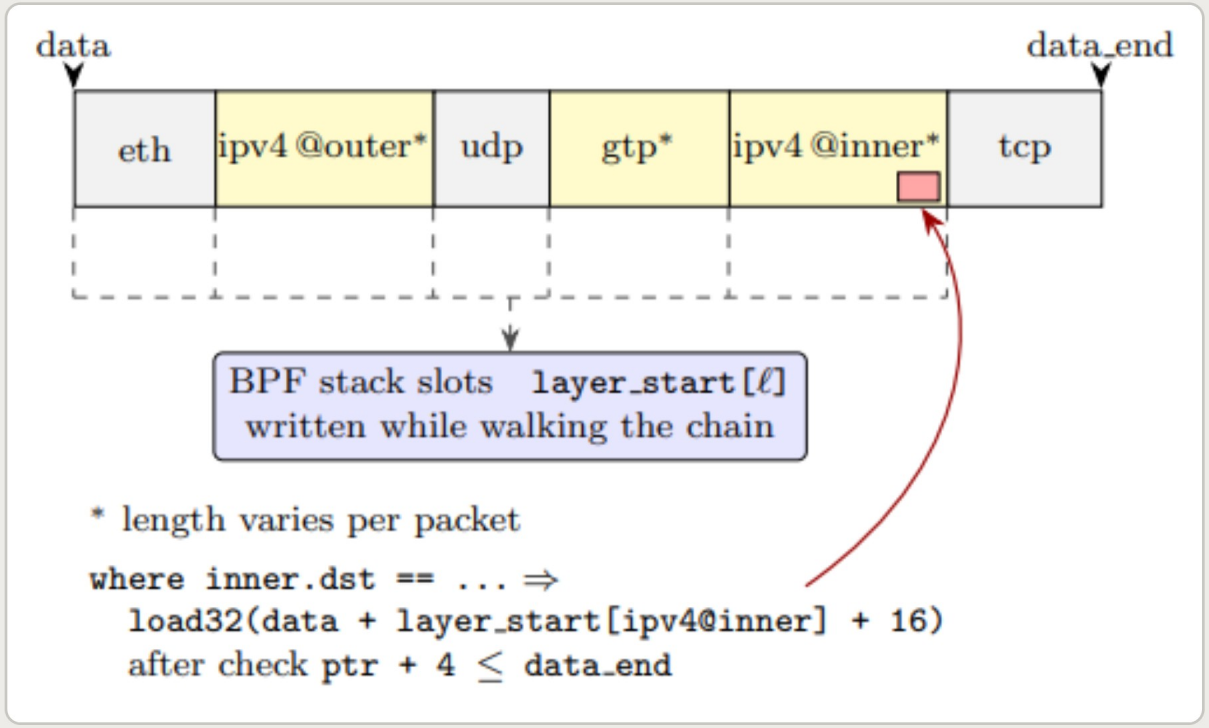
반복 횟수를 컴파일 타임에 모름

③ 중첩 캡슐화

같은 이름 헤더(outer · inner) 구분 불가

2. 제안 기법: 가변 길이 헤더

시작점 기록 (Fig. 4)



① 시작점 기록

패킷을 한 번 훑으며 각 헤더 시작점을 스택 슬롯에 기록

② 데이터 접근

기록한 시작점 + 오프셋으로 접근

접근 전 data_end 범위 검사

어긋나는 헤더 시작점을 런타임에 기록

접근 바이트코드마다 범위 검사 → **verifier-safe**

2. 제안 기법: 가변 개수 헤더

컴파일 타임에 알 수 없는 개수의 헤더 · 원소를 어떻게 인식할 것인가?

가변 개수 대상: 헤더 자체 또는 헤더의 원소(list)

각 단계(헤더 · 원소) 처리 단위 = **walk**

바깥 구조: walk들을 처리하는 바이트코드

① P4 상한 ≤ 컴파일 타임 임계값

walk를 상한 횟수만큼 펼침 (unroll)

② P4 상한 > 임계값 · 상한 미지정

bpf_loop 콜백으로 생성 · 횟수는 상수 상한 (미지정 시 컴파일러 기본값 ≠ 임계값)

상한으로 종료 보장 = verifier-safe

내부 구조: walk 하나의 구현

Algorithm 1 · 5행 : 범위 검사로 verifier-safe

Algorithm 1 Bounded walk skeleton; STOP, STRIDE, and STEP are filled in per structure.

Require: *data*, *data_end*, *base*, *cap*; STOP, STRIDE, STEP per structure

1: $r \leftarrow \perp$; $off \leftarrow base$
2: **for** $i \leftarrow 0$ **to** $cap - 1$ **do** ▷ *cap*: compile-time bound
3: **if** STOP(i, off) **then break** ▷ count reached, or end/END/malformed
4: $ptr \leftarrow data + off$
5: **if** $ptr + w > data_end$ **then break** ▷ *w*: bytes read; bounds check
6: $r \leftarrow$ STEP(ptr); **break if** r decides ▷ compare, or record option
7: $off \leftarrow off +$ STRIDE(ptr) ▷ constant, or this option's length
8: **return** r

2. 제안 기법: walk 내부 구현

패킷 구조에 따라 다른 walk 내부 구현 방식

counted walk

개수 필드 + 고정 폭 원소 · 예: SRv6 세그먼트 list

stop	i ≥ 개수 (last_entry + 1)
stride	원소 폭 상수 (16바이트)
step	원소 필드를 대상 값과 비교

TLV walk

kind로 분기 + 원소별 길이 다름 · 예: TCP 옵션 (EOL, MSS …)

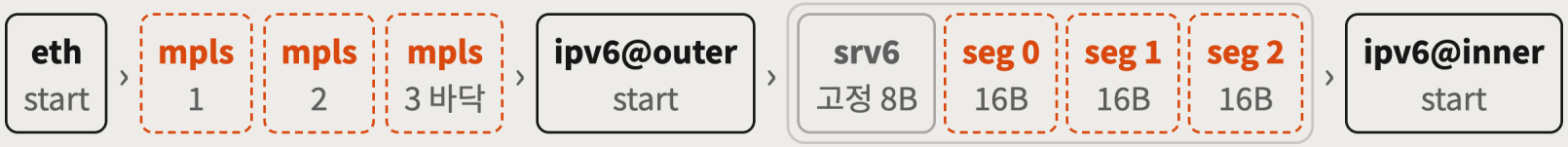
stop	영역 끝 · END · 형식 오류 · 상한
stride	그 옵션의 len
step	kind 확인 후 위치 기록, walk 뒤 where 절이 비교

sentinel chain · 헤더가 가변 개수

헤더 끝 표식까지 walk · Algorithm 1의 stop · stride · step 채우기 불필요, 상한 있는 반복 구조는 동일

2. 제안 기법: 필터 동작 예시

eth/mpls+/ipv6@outer/srv6/ipv6@inner where any(srv6.segments.addr == fc00::1) and ipv6@inner.dst == fc00::9



①	**시작점 기록**	eth 시작점 기록 (패킷 시작 = 0)
②	**헤더 개수 가변**	시작점 기록 + 바닥 표시까지 MPLS 순회 → 끝 위치 확인
③	**시작점 기록**	ipv6@outer 시작점 기록
④	**원소 개수 가변**	시작점 기록 + 개수 필드만큼 세그먼트 순회, fc00::1과 비교
⑤	**시작점 기록**	srv6 끝 위치로 ipv6@inner 시작점 기록 → inner dst = 시작점 + 24, 범위 검사 후 비교

중첩 캡슐화 같은 이름 헤더(outer · inner)는 @라벨로 구분 · 라벨 없이 반복 참조 시 거부

3. 평가: 명령어 수

평가 필터 F1–F10 (Table 2)

ID	Kunai expression	Kunai	pcap-filter	verifier
F1	`eth/ipv4/tcp where tcp.dport == 443`	199	42	8,137
F2	`eth/ipv4[src==10.0.0.0/8]/tcp where tcp.dport == 80`	209	36	8,159
F3	`eth/ipv6[src==2001:db8::/32]/tcp`	339	23	18,595
F4	`eth/vlan[tci==100]/ipv4/tcp where tcp.dport == 80`	215	51	8,177
F5	`eth/ipv4/icmp where icmp.type == 8`	85	31	315
F6	`eth/qinq/vlan/ipv4/tcp where tcp.dport == 80`	217	—	8,181
F7	`eth/ipv4@outer/udp/gtp/ipv4@inner/tcp where inner.dst == 10.0.0.1`	406	—	89,173
F8	`eth/ipv6/srv6 where any(srv6.segments.addr == fc00::1)`	333	—	3,987
F9	`eth/ipv4@outer/udp/geneve/eth/ipv4@inner/tcp where inner.dst == 10.0.0.1`	351	—	7,706
F10	`eth/ipv4/tcp where tcp.options.MSS.value == 1460`	231	—	206,733

Kunai 생성 바이트코드 명령어 수

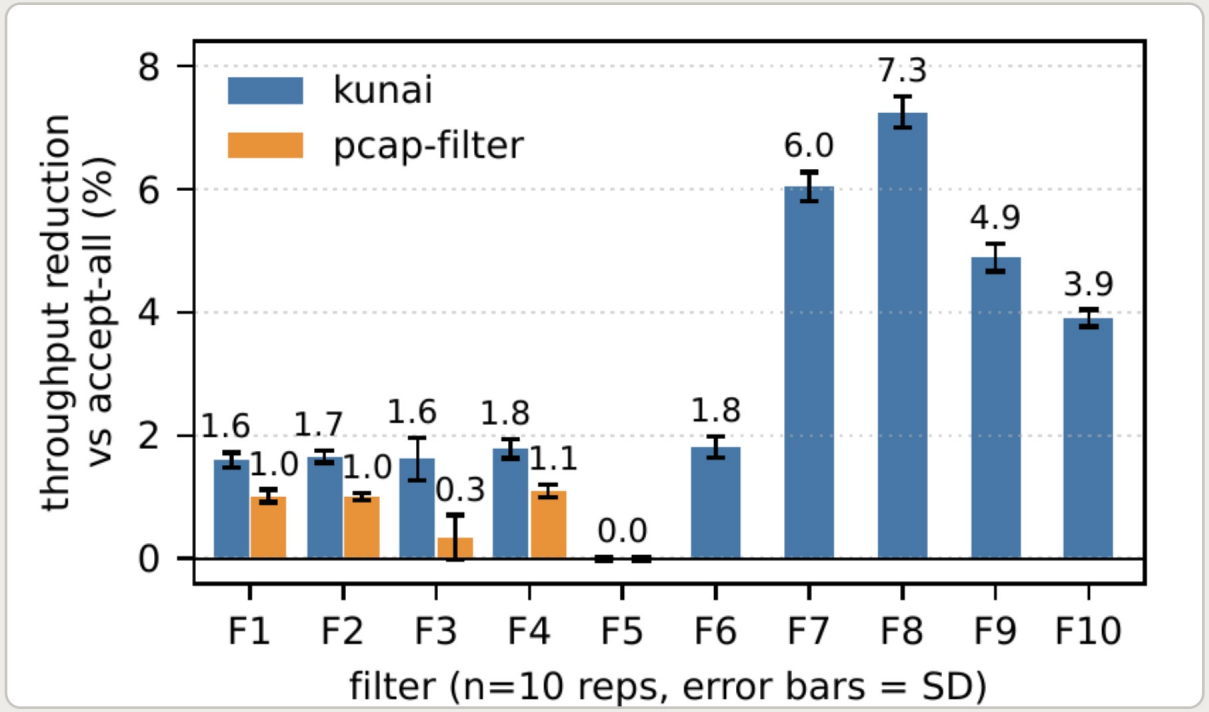
pcap-filter cBPF 경유 변환 eBPF 명령어 수

verifier 검증 시 처리한 명령어 수 = 검증기 부하

1. **pcap-filter 대비 우수한 표현력**
2. **전부 verifier 로드 성공 · 패킷 검증 결과 일치**
3. **상한 있는 반복 → 평가한 필터 모두 verifier 한도(100만) 이내 (최대 약 20만, F10)**

3. 평가: 처리율

필터 F1–F10 처리율 감소율 (Fig. 5)



baseline

복사 → ACCEPT ALL → DROP

pcap-filter

복사 → (cBPF →) eBPF 필터 → DROP

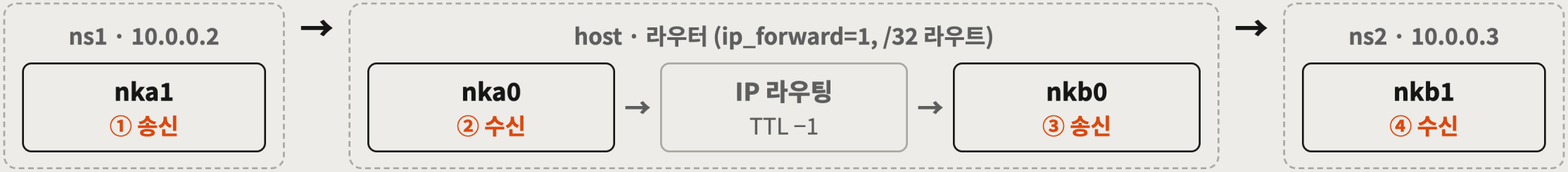
Kunai

복사 → Kunai 필터 → DROP

pcap-filter 대비 우수한 표현력과 유사한 성능

4. 추가 실험: 실험 환경

같은 ping · 한 번에 하나만 바꿔서 헤더 비교



A. veth (L2) + host 라우팅

원래 구조 (비교 기준) · Cilium 기존 방식

경로 ① → ② → ③ → ④

ARP · 실제 MAC 사용

B. netkit (L3 모드)

A에서 디바이스만 교체 · BPF 없음

경로 ① → ② → ③ → ④

ARP 없음 · MAC 미사용

C. netkit + redirect BPF

B에서 BPF만 추가

경로 ① → ③ → ④

host 수신 · 라우팅 우회

ping 1회 → ④에서 MAC · TTL 관측 (bpftrace)

4. 추가 실험: 수신 지점 헤더 (raw)

ns2 nkb1 수신 · 패킷 헤더 42바이트 (Eth 14 + IPv4 20 + ICMP 8)

	dst MAC · 6	src MAC · 6	Type · 2	IPv4 헤더 · 20	ICMP · 8
A	f64fb8903f18	a2a73f9dd1ae	0800	45000054b7d84000**3f**016fcc0a0000020a000003	0800b4043a5d0001
B	000000000000	000000000000	0800	450000549790400**3f**0190140a0000020a000003	0800490a3ae30001
C	000000000000	000000000000	0800	450000542d144000**40**01f9900a0000020a000003	08002e463b690001

MAC A 환경에서만 값 존재 (host가 링크마다 새로 쓴 MAC) · B, C는 0
Type 다음 헤더 종류라 동일 (08 00 = IPv4)
IPv4 강조 1바이트 = **TTL** (63: host 라우터가 −1 · 64: host 우회) · 나머지 차이는 ID · 체크섬
ICMP ID · 체크섬 차이

코드 · github.com/clapppp/ebpf-net-experiments